Virtual Advertising in Soccer Field:

A Comprehensive Approach to Image Processing and Banner Projection

Jannatul Ferdous, j.ferdous@student.utwente.nl, 3392120, M-EE
Pim Klumpers, p.g.h.klumpers@student.utwente.nl, 3307638, M-ROB
Oisín Barry, o.l.barry@student.utwente.nl, 3528464, M-ROB
Gautham Jeeva Kumar Suveny, g.jeevakumarsuveny@student.utwente.nl, 3400220, M-ROB

***Abstract*— This report explores the application of image processing techniques for virtual advertising in soccer fields. The study addresses the challenge of accurately projecting digital banners onto a dynamic environment while maintaining alignment with court lines. It investigates the methods used for image analysis, perspective transformation, and corner detection, ultimately aiming to enhance the viewer's experience during sports events. The findings indicate that effective integration of advanced algorithms can significantly improve the accuracy of banner placement, providing insights into future applications of virtual advertising in sports.**

***Index Terms*— Advertisement, Image Processing, Perspective Transformation, Soccer Field.**

I. INTRODUCTION

Virtual advertising in sports has gained significant attention in recent years as a method to enhance viewer engagement and generate additional revenue [1]. The rapid advancement of image processing technologies enables the seamless integration of digital content into live sports broadcasts, creating immersive experiences for audiences [2]. This report addresses the challenges associated with projecting virtual advertisements onto soccer fields, particularly in scenarios where camera angles and environmental conditions can vary significantly [3].

The primary problem explored in this report is the effective placement and projection of virtual advertisements on a soccer field while ensuring that they maintain visual integrity and alignment with the field's geometry. The research question guiding this study is: *How can image processing techniques be utilized to accurately project virtual advertisements onto a soccer field in real-time?*

This question leads to several sub-questions:

1. What are the key image processing methods that can facilitate accurate detection and alignment of field lines?
2. How can perspective transformation be applied to ensure the advertisement appears correctly positioned from different camera angles?
3. What are the limitations of current methods in handling variations in lighting and field markings?

II. METHODS AND MATERIALS

A. Materials

1) Hardware:

The processing tasks were executed in both the Google Colab environment, and in environments running on personal high-powered laptops. The Google Colab environment uses the GPU runtime with an NVIDIA Tesla T4 GPU, providing access to 16 GB of GPU memory. Both environments enable efficient handling of computationally intensive tasks like image segmentation, edge detection, and homography transformations.

2) Software:

***Programming Language*:**

Python was used for coding the algorithms due to its extensive libraries for image and video processing. We used both a local python environment and Google Colab, an online Python environment, to facilitate coding and computation for the project.

***Libraries and Tools*:**

***OpenCV*:**

For image and video processing tasks, including color segmentation, edge detection, line detection, and perspective transformation.

***NumPy*:**

For numerical operations, specifically for managing matrices and arrays essential for proper data management, homography calculations and point transformations.

Matplotlib:

Used for displaying frames with detected features and advertisement overlays.

3) Data:

Video Clip:

Selected soccer field videos with 1080p resolution, ensuring clear capture of field lines and goal posts from various perspectives. The video clip used is 20 seconds long and consists of 4 stitched together video fragments where the camera is pointed towards the goal. The shots used have minimal zoom and a panoramic perspective to maintain stable reference points across frames. These 4 video fragments were taken from the same soccer match to improve consistency.

Reference Image:

A reference image is used to provide a reference of both the field and the points to track in the video frame. The chosen reference image provides an ideal perspective of the soccer field, capturing the full layout and boundaries necessary for accurate homography. Key points, labeled 0 through 5, represent the specific corners used in the homography calculation. The magenta cross marks on the field indicate the locations designated for virtual advertisement placement, ensuring that the projected content aligns with the field's perspective.

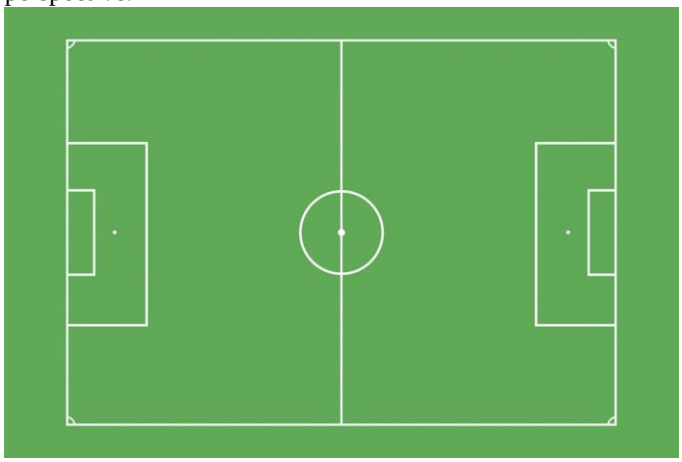


Figure 1: *Reference Image of Soccer Field for Homography.*

Advertisement image:

The advertisement image selected for this project features a logo, with a resolution of 504 x 153 pixels, allowing it to maintain clarity when scaled and transformed onto the soccer field.



Figure 2: *Advertisement Image Featuring the Logo.*

B. Methods

1) Overview of the problem:

In this project, the input consists of images or video frames from video clips of a soccer field, while the desired output is the successful placement of virtual advertisements on the field within each frame. The algorithm must effectively run through all the video clips to achieve consistent advertising projection across varying frames.

2) Key Challenges and Solutions:

Accurate Color Segmentation:

Issue: The varied shades of green on the soccer field made it difficult to effectively segment the green color

Solution: Implemented a robust color segmentation function using HSV color space to isolate green areas accurately. This involved defining a range for lower and upper green values, which effectively captured the field's color variations while excluding unwanted hues.

Line Detection in Noisy Environments:

Issue: Edge detection was hampered by noise and other artifacts in the image.

Solution: Applied a Gaussian blur before using the Canny edge detector to smooth out the image and reduce noise. Furthermore, morphological operations were introduced to enhance the quality of the detected edges, facilitating better line detection through Hough Line Transform.

Identifying and Categorizing Lines:

Issue: Differentiating between parallel and perpendicular lines posed a challenge due to varying angles and overlaps**.**

Solution: Developed a systematic approach to calculate angles between detected lines. This allowed for the categorization of lines based on their relative angles in reference to the longest line, enabling the identification of parallel lines (within a specified threshold) and perpendicular lines (defined by a range).

Accurate Corner Detection:

Issue: The existing corner detection was prone to false positives due to closely spaced lines and noise.

Solution: Introduced a filtering mechanism that grouped similar lines based on angle similarity and proximity. This reduced false corner detections by ensuring that only intersections of significant lines were considered as valid corner points. Besides finding the corresponding corners related to the reference image points, we also implemented a weighted average averaged the corners detected in two frames to further decrease the likelihood of false corner detection.

Camera Calibration Parameters:

For this problem, we planned to use camera calibration to get the focal length which was planned to be used in rotating the advertisement plane by a specified angle to appear like a card that props out of the place of the pitch. This would have to be done by using a 3D rotation and then reprojecting the advertisement panels points into the 2D plane. This rotation would also have to be transformed with respect to the camera rotation as well such that the advertisement would face the same direction.

3) Strategy to Obtain Output from Input

The algorithm comprises several steps to achieve the desired outcome of overlaying advertisements on the soccer field. Below is a detailed explanation of each step involved in the process (see Figure 3 for the pseudocode representation of the algorithm):

i. **Input acquisition:**

At this stage, the single frame image from the video clip served as the input, with the reference image stored to indicate the desired advertisement placement.

ii. **Color segmentation**:

This step involved applying color segmentation to isolate green regions corresponding to the soccer field, effectively reducing background noise. This step was also done separately to isolate white areas, effectively finding the goalpost, to be used as a reference point later in the labelling section.

iii. **Masking out areas**:

Morphological operations, such as dilation and erosion, were then performed to create a mask that retained only the segmented field area, removing irrelevant background regions.

iv. **Edge detection**:

Next, Canny edge detection was applied to the masked image, identifying edges essential for recognizing the field's boundaries and markings.

v. **Morphological operations**:

Morphological techniques further refined the edges, enhancing line continuity and connectivity, which improved the quality of detected lines for subsequent steps.

vi. **Line detection (Hough transform)**:

The Hough Line Transform was then used to detect straight lines within the edge-detected image, facilitating the identification of key boundaries on the soccer field.

vii. **Line filtering**:

The Hough transform gives a lot of lines that are overlapping and thus represent the same line. To filter them, all lines assessed according to angle and position, which were used to group the overlapping lines belonging to the same line. The minima and maxima of these groups where then taken to get the total span of the lines described by the line groups. This results in a list of only the lines spanning the field.

viii. **Corner detection**:

Intersections between filtered lines were analyzed to find key corner points on the field, which will serve as reference points for advertisement placement.

ix. **Corner labelling**:

The centre of the crossbar, found using steps iii – vi on the white segmented image, was used as a reference point to label the detected corners, based on their orientation and distance from the point.

x. **Homography transformation**:

The homography matrix is computed by using the reference image as a template, we manually mark points in the reference image that correspond to corners where the lines intersect. The same corresponding points are selected in the frames and homography matrix is computed for it. This is used to wrap the advertisement on the pitch at another reference point.

xi. **Advertisement placement**:

Using the calculated homography, the advertisement's coordinates were transformed, positioning it appropriately on the soccer field. This has been achieved using OpenCV's warpPerspective function.

xii. **Output generation**:

Finally, the processed frames with the transformed advertisement was planned to be overlaid on the pitch to create the final output.

```
FUNCTION main(input_video_file, ad_image,
reference_image_corners):

    // Step 1: Acquire frames from the video
source
    frames = acquire_images(input_video_file)
    processed_frames = []

    // Step 2: Process each frame
    FOR each frame IN frames DO

        // Step 3: Apply green color segmentation
        I_seg = color_segmentation_green(frame)

        // Step 4: Mask out the outside area to
reduce background noise
        TestMask = create_mask(I_seg)  // Apply
morphological operations to create a mask
        I_seg = apply_mask(I_seg, TestMask)

        // Step 5: Detect edges in the masked
segmented image
        I_edges = detect_edges(I_seg)
// Step 6: Perform morphological operations on
the edges
        dilated_edges = dilate_edges(I_edges)
// Dilation operation to thicken edges
        eroded_edges = erode_edges(dilated_edges)
// Erosion operation to refine edges

        // Step 7: Perform Hough Line
Transformation
        lines = detect_hough_lines(eroded_edges)

        // Step 8: Find the longest line among
detected lines
        longest_line = find_longest_line(lines)

        // Step 9: Draw a combined image with
both parallel and perpendicular lines
        combined_image =
find_and_draw_parallel_perpendicular(frame,
lines, longest_line)

// Step 10: Detect corners from the detected
lines
        CornerPoints = corner_detection(lines)

        // Step 11: Calculate the homography
matrix using detected corners
        H = calculate_homography(CornerPoints,
reference_image_corners)

        // Step 12: Place the advertisement using
the homography matrix
        A_trans = warp_advertisement(ad_image, H)

        // Step 13: Generate the output frame
with the advertisement overlay
        output_frame =
generate_output(combined_image, A_trans)

        // Step 14: Append the processed frame to
the list
        processed_frames.append(output_frame)

    // Step 15: Return the processed frames
    RETURN processed_frames

END FUNCTION
```

Figure 3: *Pseudocode for the Advertisement Placement Algorithm.*

4) Performance evaluation:

The performance of the virtual advertisement overlay algorithm was evaluated using several criteria to ensure accurate placement and visual integrity of the advertisements throughout the soccer field video frames. The evaluation comprised both qualitative observations and quantitative metrics.

Qualitative Assessment

Visual Inspection: Processed video frames were visually examined to confirm that the advertisements were seamlessly integrated within the soccer field context. Observers assessed the alignment of advertisements with field lines, the clarity of the ad imagery, and the overall realism.

User Feedback: Feedback was collected from group members who viewed the output videos to gauge their perceptions of the advertisement placements. Comments focused on how naturally the advertisements appeared in relation to the soccer field.

Quantitative Metrics

Accuracy of Homography: The accuracy of homography is checked using the transformation of the advertisement. This was also trivial in highlighting the issues with the point corresponcences.

Frame Processing Time: The time taken to process each frame was recorded to assess the algorithm's efficiency. This is crucial for real-time applications, where a low processing time per frame is desirable. In the Google Colab environment, it took a maximum of 25 minutes to process a 32-second video.

III. Results

In this section, we present the results of the processing pipeline, demonstrating the transformations applied to the input image/frame at each significant stage. The following images illustrate the progression from the original input to the final output with the advertisement placed on the soccer field.

1) Output images:

i. Input frame:



Figure 4: *Original input image captured from the soccer field, serving as the basis for further processing.*

ii. Color segmentation:



Figure 5: *Color segmentation applied to isolate the green regions corresponding to the soccer field.*

iii. Area mask:



Figure 6: *Area mask for taking out the noise outside of the playing field*

iv. Masked color segmentation:



Figure 7: *Resulting mask after performing morphological operations to retain only the segmented field area while removing irrelevant background regions.*

v. Edge detection:

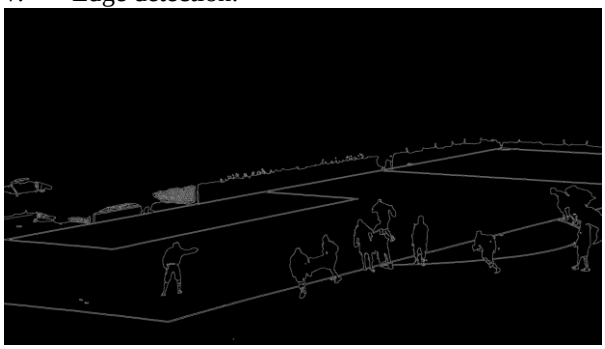


Figure 8: *Image after applying Canny edge detection, highlighting the edges crucial for recognizing the field's boundaries and markings.*

vi. Morphological operations:

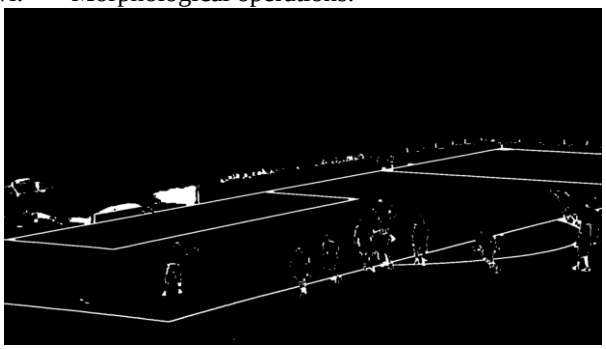


Figure 9: *Further refined edges using morphological operations, enhancing line continuity and connectivity for improved line detection.*

vii. Line detection (Hough transform):

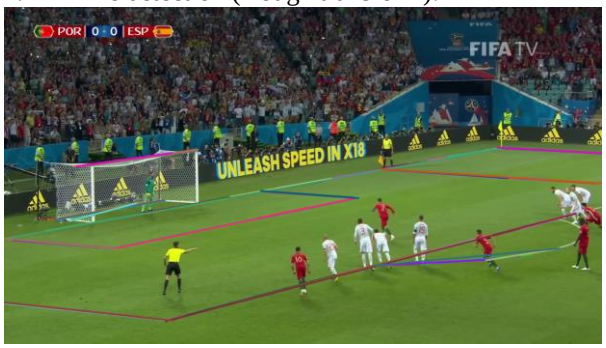


Figure 10: *Detected straight lines in overlaid on the edge-detected image, identifying the key boundaries of the soccer field.*

viii. Line filtering:



Figure 11: Highlighted blue lines showing all the parallel and perpendicular lines detected in relation to the longest line.

ix. Corner detection:

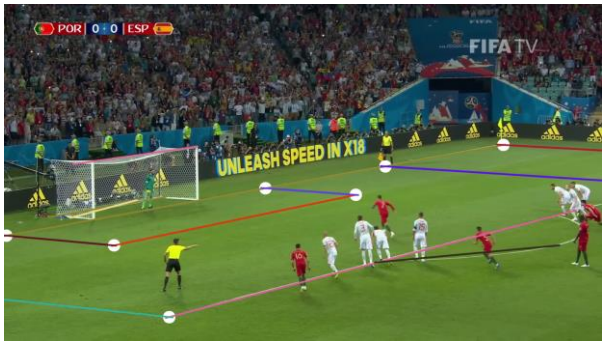


Figure 12: *Detected corner points in white at the intersections of lines.*

x. Corner identification:



Figure 13: *Identified corner points in accordance with the reference image shown n figure 1*

xi. Homography transform:

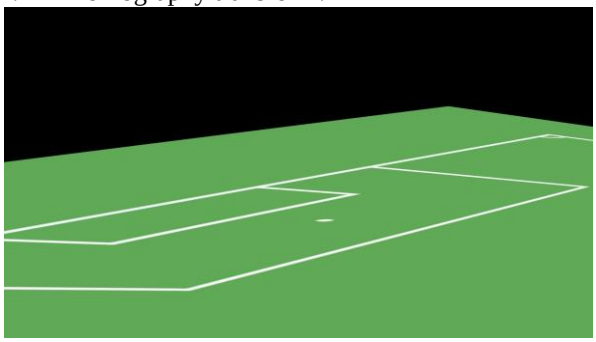


Figure 14: *Visualization of the mapping from detected corners to reference points for the homography calculation.*

xii. Advertisement placement:



Figure 15: *Final output image with the advertisement overlaid, positioned on the soccer field according to the homography transformation.*

2) Output stability and quality:

The output for step ii through v proved to be very consistent and therefore had good stability, which in all provided a good edge detection of the lines in the frame. Step vi proved to be the first part where the stability and quality of the image went down slightly, as there were still some noise remnants from the players and the edge of the field where the image was masked. The results of steps vii and viii gave filtered lines that closely adhered to the lines within the input frame. Some stability issues were observed in the areas where step vi introduced some noise. Step ix of detecting corners showed to give corners which qualities and stabilities were good, but once again slightly affected by the noise in the lines which was introduced in step vi. Step x was able to identify the corners but proved to have difficulties in the consistencies of this process, where it often identified the corners incorrectly. Steps xi and xii of performing the homography transform and placing the image proved to have major problems in both quality and stability of placing the advertisement.

IV. Discussion

The result shows that we were unsuccessful in developing a program that can place the advertisement image on the soccer pitch. Although the line detection was working as expected, the challenge was in associating the points with corresponding points in the reference image. This was worsened by the soccer pitch having lines all in an arbitrary angle from the camera's viewpoint and the homography computation being dependent on more points to optimize using RANSAC. Having only a few points as an input to homography also poses a challenge in the accuracy of it.

V. Conclusion

This paper focused on creating a program capable of overlaying a virtual advertisement over a video of a soccer field. The process was split up into different steps that would start with a video frame, filter it and get the edges within it, use line detection ad their overlap to det the corners of the playing field, and use these corners with a homography transform to place a virtual advertisement on the field. The results, however, show that we were unsuccessful in developing a program capable of overlaying virtual advertisements on a soccer field. Various steps of the program are shown to have potential to get to a successful result, but the small issues in line detection, corner detection, corner identification and homography transformation added up to such a degree that the program was unable to successfully place the advertisement.

APPENDIX C

PYTHON CODE FOR THE IMAGE PROCESSING AND BANNER PROJECTION PROCESS

```
import cv2 as cv
import numpy as np
import math
from operator import itemgetter
#from matplotlib import pyplot as plt
#import sys



REF_PT_MAP = {
    (6, 1) : [100, 50],
    (4, 8) : [100, 210],
    (2, 8) : [100, 285],
    (3, 8) : [140, 285],
    (1, 6) : [140, 415],
    (2, 5) : [100, 415],
    (2, 7) : [220, 492]
}

obj_pts = [
    [100, 285],             # GL0
    [140, 285],             # GL1
    [140, 415],             # GL2
    [100, 415],             # GL3
    [100, 210],             # OB0
    [100, 50]               # P0
]

MIN_MATCH_COUNT = 10
color = np.random.randint(0, 255, (500, 3))
PreviousPoints = []
PreviousPoints2 = []
PreviousPoints3 = []
PreviousPoints4 = []

setPoints = []
setPoint = [0, 0, 0, 0]
setPoints.append(setPoint)

goalLengths = []
goalPostLength = 1
goalLengths.append(goalPostLength)

#CHECK
PATH!!!
```

```
    return edges, gray

# Function to detect lines using Hough Line Transform
def detect_hough_lines(edges):
    lines = cv.HoughLinesP(edges, 1, np.pi / 180, threshold=200, minLineLength=150, maxLineGap=20)
    return lines

# Function for color segmentation (only green color)
def color_segmentation_green(image):
    hsv_image = cv.cvtColor(image, cv.COLOR_BGR2HSV)
    lower_green = np.array([40, 40, 40])
    upper_green = np.array([50, 255, 255])
    mask_green = cv.inRange(hsv_image, lower_green, upper_green)
    segmented_image = cv.bitwise_and(image, image, mask=mask_green)
    return segmented_image

#Function for applying a saturation mask to a frame
def increase_image_saturation(image, saturation = 2):
    imghsv = cv.cvtColor(image, cv.COLOR_BGR2HSV).astype("float32")
    (h, s, v) = cv.split(imghsv)
    s = s*saturation
    s = np.clip(s,0,255)
    imghsv = cv.merge([h,s,v])
    saturatedImage = cv.cvtColor(imghsv.astype("uint8"), cv.COLOR_HSV2BGR)
    return saturatedImage

#Function for applying a contrast and brightness mask to a frame
def change_image_brightness_contrast(image, brightness, contrast):
    editedImage = cv.addWeighted(image, contrast, np.zeros(TestImage.shape, TestImage.dtype), 0,
brightness)
    return editedImage

#Function for applying a deletion mask to a color segmented image
def delete_outside_area(image):
    kernel = cv.getStructuringElement(cv.MORPH_ELLIPSE, (6, 3))
    TestMask = cv.erode(image, kernel, iterations=15)
    TestMask = cv.cvtColor(TestMask, cv.COLOR_BGR2GRAY)
    empty, TestMask = cv.threshold(TestMask, 0, 255, cv.THRESH_BINARY)
    TestMask = cv.dilate(TestMask, kernel, iterations=30)
    TestMask = cv.cvtColor(TestMask, cv.COLOR_GRAY2BGR)
    filteredimage = cv.bitwise_and(image, TestMask)
    return filteredimage

# Function to calculate the length of a line
def calculate_line_length(line):
    x1, y1, x2, y2 = line[0]
    return np.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

# Function to calculate the angle of a line
def calculate_line_angle(line):
    x1, y1, x2, y2 = line #was [0]
    angle_radians = np.arctan2(y2 - y1, x2 - x1)
    angle_degrees = np.degrees(angle_radians)
    return angle_degrees

# Function to find and draw parallel and perpendicular lines with angle thresholds
def find_and_draw_parallel_perpendicular(image, lines, longest_line):
    if longest_line is None:
        return None

    longest_angle = calculate_line_angle(longest_line)
    parallel_threshold_low = 0
    parallel_threshold_high = 6
    perpendicular_threshold_low = 70
    perpendicular_threshold_high = 80
    parallel_length_threshold = 250
    perpendicular_length_threshold = 150

    combined_image = image.copy()

    for line in lines:
        angle = calculate_line_angle(line)
        length = calculate_line_length(line)
```

```
        angle_relative = abs(angle - longest_angle) % 180

        if (parallel_threshold_low <= angle_relative <= parallel_threshold_high) and length >
parallel_length_threshold:
            cv.line(combined_image, tuple(line[0][:2]), tuple(line[0][2:]), (0, 255, 0), 2)

        if (perpendicular_threshold_low <= abs(angle_relative - 90) <= perpendicular_threshold_high) and
length > perpendicular_length_threshold:
            cv.line(combined_image, tuple(line[0][:2]), tuple(line[0][2:]), (0, 0, 255), 2)

    return combined_image

def getMaxLines(edges):

        # Step 3: Morphological operations (JANNAT) customised
        kernel = cv.getStructuringElement(cv.MORPH_RECT, (5, 5))
        dilated_edges = cv.dilate(edges, kernel, iterations=1)
        kernel = cv.getStructuringElement(cv.MORPH_RECT, (6, 6))
        eroded_edges = cv.erode(dilated_edges, kernel, iterations=1)
        kernel = cv.getStructuringElement(cv.MORPH_RECT, (2, 2))
        eroded_edges = cv.dilate(eroded_edges, kernel, iterations=1)

        # Step 4: Detect lines using Hough Line Transform (JANNAT)
        lines = detect_hough_lines(eroded_edges)
        hough_image = TestImage.copy()

        if lines is not None:
            if lines.all():
                for i in range(0, len(lines)):
                    l = lines[i][0]
                    cv.line(hough_image, (l[0], l[1]), (l[2], l[3]), color[i].tolist(), 3, cv.LINE_AA)

        #Values for filtering and grouping
        minLength = 10
        minRes = 1
        maxAngleDifference = 5
        maxDistanceDifference = 15

        #Filtering and grouping lines
        skipIndex = []
        LinesSorted = []
        if lines is not None:

            #Load first line
            for i, line1 in enumerate(lines):
                skipLoop = False
                line1 = line1[0]
                v1 = line1[2:4] - line1[0:2]
                len1 = np.linalg.norm(v1)

                for s in range(0, len(skipIndex)):
                    skip = skipIndex[s]
                    if skip == i:
                        skipLoop = True
                        break

                #Skip it its too short or if it has allready been appended
                if len1 < minRes * minLength or skipLoop == True:
                    continue

                #Load second line
                LinesSimmilar = []
                LinesSimmilar.append(line1)
                for j, line2 in enumerate(lines):
                    skipLoop = False
                    line2 = line2[0]
                    if i == j:
                        continue
                    v2 = line2[2:4] - line2[0:2]
                    len2 = np.linalg.norm(v2)

                    for s in range(0, len(skipIndex)):
```

```
                        skip = skipIndex[s]
                        if skip == j:
                            skipLoop = True
                            break

                    #Skip it its too short or if it has allready been appended
                    if len2 < minRes * minLength or skipLoop == True:
                        continue

                    #Skips line if its not ~parallel
                    angleDifference = abs( calculate_line_angle(line1) - calculate_line_angle(line2) )
                    if angleDifference > maxAngleDifference:
                        continue

                    #Get line points (laid it out for personaly clarity)
                    x1 = line1[0]
                    y1 = line1[1]
                    x2 = line1[2]
                    y2 = line1[3]
                    x3 = line2[0]
                    y3 = line2[1]
                    x4 = line2[2]
                    y4 = line2[3]

                    #Gives line as Ax+By=C format
                    def line(p1, p2):
                        A = (p1[1] - p2[1])
                        B = (p2[0] - p1[0])
                        C = (p1[0]*p2[1] - p2[0]*p1[1])
                        return A, B, -C

                    #Get first line as Ax+By=C format
                    L1 = line([x1, y1], [x2, y2])

                    #Gives y of an Ax+By=C line by inputting x
                    def getY(line, x):
                        y = ((line[2])-(line[0]*x))/line[1]
                        return y

                    #Input both x coords of second line. Checks if y of the extended first line meets the y
of the second line
                    if abs(getY(L1, x3) - y3) > maxDistanceDifference and abs(getY(L1, x4) - y4) >
maxDistanceDifference:
                        continue

                    #If on same line, group together
                    skipIndex.append(j)
                    LinesSimmilar.append(line2)

                #Append each group in an array of groups
                LinesSorted.append(LinesSimmilar)



        #Go through all line groups, get min and max point (Line that spans all lines) and append
        LineFiltered = []
        for i in range(0, len(LinesSorted)):
            sorted = LinesSorted[i]

            #Initialise variables
            lineOut = sorted[0]
            minX1 = sorted[0][0]
            minY1 = sorted[0][1]
            maxX2 = sorted[0][2]
            maxY2 = sorted[0][3]

            #Gather min and max for the line group
            for j in range(0, len(sorted)):
                line = sorted[j]

                #Keep track of the mimima and maxima
                if calculate_line_angle(line) >= 0:
```

```
                    if min(line[0], line[2]) < minX1:
                        minX1 = min(line[0], line[2])
                    if max(line[0], line[2]) > maxX2:
                        maxX2 = max(line[0], line[2])
                    if min(line[1], line[3]) < minY1:
                        minY1 = min(line[1], line[3])
                    if max(line[1], line[3]) > maxY2:
                        maxY2 = max(line[1], line[3])
                if calculate_line_angle(line) < 0:
                    if min(line[0], line[2]) < minX1:
                        minX1 = min(line[0], line[2])
                    if max(line[0], line[2]) > maxX2:
                        maxX2 = max(line[0], line[2])
                    if min(line[1], line[3]) < maxY2:
                        maxY2 = min(line[1], line[3])
                    if max(line[1], line[3]) > minY1:
                        minY1 = max(line[1], line[3])

            #Fill values and append
            lineOut[0] = minX1
            lineOut[1] = minY1
            lineOut[2] = maxX2
            lineOut[3] = maxY2
            LineFiltered.append(lineOut)

        return LineFiltered

#Get intersection points. Extend lines by 75px and check if they intersect within bounds
def get_intersection_points(LineFiltered):
    lineExtension = 75
    CornerPoints = []
    if LineFiltered is not None:

        #Get first line and extend
        for i in range(0, len(LineFiltered)):
            line1 = LineFiltered[i]
            line1 = line1
            ab = line1[2:4] - line1[0:2]
            v = (ab / np.linalg.norm(ab)) * lineExtension
            line1Ext = [line1[0:2] - v, line1[2:4] + v]

            #Get second line and extend
            for j in range(0, len(LineFiltered)):
                line2 = LineFiltered[j]
                ab = line2[2:4] - line2[0:2]
                v = (ab / np.linalg.norm(ab)) * lineExtension
                line2Ext = [line2[0:2] - v, line2[2:4] + v]

                #Get line as Ax+By=C format
                def line(p1, p2):
                    A = (p1[1] - p2[1])
                    B = (p2[0] - p1[0])
                    C = (p1[0]*p2[1] - p2[0]*p1[1])
                    return A, B, -C

                #Get intersection coords using determinant (if no intersection, gives [-1, -1])
                def intersection(L1, L2):
                    D  = L1[0] * L2[1] - L1[1] * L2[0]
                    Dx = L1[2] * L2[1] - L1[1] * L2[2]
                    Dy = L1[0] * L2[2] - L1[2] * L2[0]
                    if D != 0:
                        x = Dx / D
                        y = Dy / D
                        return round(x),round(y)
                    else:
                        return -1, -1

                #Get intersection point from the two lines
                L1 = line(line1Ext[0], line1Ext[1])
                L2 = line(line2Ext[0], line2Ext[1])
                R = intersection(L1, L2)

                #Calculate all the bounds and anges which are used to filter
```

```
                bound1 = ((line1Ext[0][0] <= R[0] <= line1Ext[1][0]) or (line1Ext[0][0] >= R[0] >=
line1Ext[1][0]))
                bound2 = ((line2Ext[0][0] <= R[0] <= line2Ext[1][0]) or (line2Ext[0][0] >= R[0] >=
line2Ext[1][0]))
                bound3 = ((line1Ext[0][1] <= R[1] <= line1Ext[1][1]) or (line1Ext[0][1] >= R[1] >=
line1Ext[1][1]))
                bound4 = ((line2Ext[0][1] <= R[1] <= line2Ext[1][1]) or (line2Ext[0][1] >= R[1] >=
line2Ext[1][1]))
                angle1 = calculate_line_angle(line1)
                angle2 = calculate_line_angle(line2)
                angleDifference = abs( max(angle1, angle2) - min(angle1, angle2) )

                height, width = OrigImage.shape[:2]

                #If exists, within bounds, at expected angle within lines and within frame, appends found
corner to list
                if R and bound1 and bound2 and bound3 and bound4 and angleDifference > 12 and 0 <= R[0] <=
width and 0 <= R[1] <= height:
                    append = True
                    #Make sure not to append corners allready included (can be optimised but im too tired
atm lol)
                    for k in range(0, len(CornerPoints)):
                        if CornerPoints[k] == R:
                            append = False
                    if append == True:
                        CornerPoints.append(R)

    return CornerPoints

#Compare the points to the previous frame, use only the ones that were there before (within 50px)
def average_strong_corners(CornerPoints):
    differenceXYmax = 50
    AveragePoints = []

    #Make sure to use global variables
    global PreviousPoints
    global PreviousPoints2
    global PreviousPoints3
    global PreviousPoints4

    if CornerPoints is not None:
        #Loop through all current points
        for i in range(0, len(CornerPoints)):
            point1 = CornerPoints[i]

            #Loop through frame-1 points
            for j in range(0, len(PreviousPoints)):
                point2 = PreviousPoints[j]
                xdiff = abs(point1[0] - point2[0])
                ydiff = abs(point1[1] - point2[1])

                #If current frame and frame-1 have the same relative points
                if xdiff < differenceXYmax and ydiff < differenceXYmax:
                    #Loop through frame-2 points
                    for k in range(0, len(PreviousPoints2)):
                        point3 = PreviousPoints2[k]
                        xdiff = abs(point2[0] - point3[0])
                        ydiff = abs(point2[1] - point3[1])

                        #If frame-1 and frame-2 have the same relative points
                        if xdiff < differenceXYmax and ydiff < differenceXYmax:
                            #Loop through frame-3 points
                            for l in range(0, len(PreviousPoints3)):
                                point4 = PreviousPoints3[l]
                                xdiff = abs(point3[0] - point4[0])
                                ydiff = abs(point3[1] - point4[1])

                                #If frame-2 and frame-3 have the same relative points
                                if xdiff < differenceXYmax and ydiff < differenceXYmax:
                                    #Loop through frame-4 points
                                    for m in range(0, len(PreviousPoints4)):
                                        point5 = PreviousPoints4[m]
                                        xdiff = abs(point4[0] - point5[0])
```

```
                                        ydiff = abs(point4[1] - point5[1])

                                        #If frame-3 and frame-4 have the same relative points
                                        if xdiff < differenceXYmax and ydiff < differenceXYmax:
                                            #Strong point sequence found, apply weighted average over all
op the points in sequence
                                            xavg = round((point1[0]*4 + point2[0]*4 + point3[0]*3 +
point4[0]*3 + point5[0]*2)/16)
                                            yavg = round((point1[1]*4 + point2[1]*4 + point3[1]*3 +
point4[1]*3 + point5[1]*2)/16)

                                            #Add point to list
                                            AveragePoints.append([xavg, yavg])

    #Update all previous points
    PreviousPoints4 = PreviousPoints3
    PreviousPoints3 = PreviousPoints2
    PreviousPoints2 = PreviousPoints
    PreviousPoints = CornerPoints

    return AveragePoints

# Optional function to mark points on pitch and show relative distance and octant
def mark_pts(im, pts, marker_color = (255, 50, 0), text_color = (0, 50, 255), marker_type =
cv.MARKER_CROSS, text_sz = 2, marker_sz = 30):
    for i, p in enumerate(pts):
        cv.drawMarker(im, (int(p[0]), int(p[1])), marker_color, marker_type, marker_sz, 3)
        cv.putText(im, (str(p[2]) + ", " + str(int(p[3]))), (int(p[0]), int(p[1] - 10)),
cv.FONT_HERSHEY_COMPLEX, text_sz, text_color, 2)
    return im

def rescale_frame(frame_input, percent=75):
    width = int(frame_input.shape[1] * percent / 100)
    height = int(frame_input.shape[0] * percent / 100)
    dim = (width, height)
    return cv.resize(frame_input, dim, interpolation=cv.INTER_AREA)

# Get point at centre of crossbar and its length
def get_goal_reference(LineGoal):
        if LineGoal:
            # sort based on y coord to pick correct marker point
            LineGoal.sort(key=itemgetter(1))

            markerPoint = 0

            # update lists of previous points/lengths for use in averaging
            setPoints.append(LineGoal[markerPoint])
            goalLengths.append(math.dist((LineGoal[markerPoint][0], LineGoal[markerPoint][1]),
(LineGoal[markerPoint][2], LineGoal[markerPoint][3])))

            # only keep track of last num of points/lengths
            if len(setPoints) > 10:
                setPoints.pop(0)
                goalLengths.pop(0)

            avgX_1 = 0
            avgY_1 = 0
            avgX_2 = 0
            avgY_2 = 0
            avgLen = 0
            for i in range(0, len(setPoints)):
                avgX_1 += setPoints[i][0]
                avgY_1 += setPoints[i][1]
                avgX_2 += setPoints[i][2]
                avgY_2 += setPoints[i][3]
                avgLen += goalLengths[i]

            # get average of end points of crossbar to get centre
            bottomGoalPoint = (int(avgX_1 / len(setPoints)), int(avgY_1 / len(setPoints)))
            topGoalPoint = (int(avgX_2 / len(setPoints)), int(avgY_2 / len(setPoints)))

            centreX = (topGoalPoint[0] + bottomGoalPoint[0]) // 2
```

```
            centreY = (topGoalPoint[1] + bottomGoalPoint[1]) // 2

            setPoint = bottomGoalPoint
            goalPostLength = avgLen / len(goalLengths)
            cv.drawMarker(OrigImage, setPoint, (0, 50, 255), cv.MARKER_SQUARE, 30, 3)
            return setPoint, goalPostLength
        return (0, 0), 1
# Takes in array of points, dest 4D array, setPoint and goalPostLength
# calcs both distance from each point to setPoint and which 'octant'
# the point is taking the setPoint as the origin and adds to new array
def calc_dist_orient(AveragePoints, averagePointsDistance, setPoint, goalPostLength):
        # Iterate over each point in AveragePoints and calculate the distance
        for l in AveragePoints:
            # Calculate the distance from setPoint
            dist = math.dist(setPoint, l)
            relDist = dist // (goalPostLength / 2)

            # calculate angle between two points taking setpoint as origin
            # equalise points compared to set point
            tempY = l[1] - setPoint[1]
            tempX = l[0] - setPoint[0]

            rads = math.atan2(tempY, tempX) / math.pi
            degrees = math.degrees(math.atan2(tempY, tempX))
            if degrees < 0:
                degrees += 360

            octant = 8 - ( degrees // 45 )

            # Concatenate the original point with the distance to form a new array with three elements
            new_point = np.concatenate((l, [relDist, octant]))

            # Append the new point to the list
            averagePointsDistance.append(new_point)
        return averagePointsDistance



# # Annotates points with index in an image
# def mark_pts(im, pts, marker_color = (255, 50, 0), text_color = (0, 50, 255), marker_type =
cv.MARKER_CROSS, text_sz = 2, marker_sz = 30):
#     for i, p in enumerate(pts):
#         cv.drawMarker(im, (int(p[0]), int(p[1])), marker_color, marker_type, marker_sz, 3)
#         cv.putText(im, str(int(p[3])), (int(p[0]), int(p[1] - 10)), cv.FONT_HERSHEY_COMPLEX, text_sz,
text_color, 2)
#     return im



# scales down a frame to a target percentage quality
def rescale_frame(frame_input, percent=75):
    width = int(frame_input.shape[1] * percent / 100)
    height = int(frame_input.shape[0] * percent / 100)
    dim = (width, height)
    return cv.resize(frame_input, dim, interpolation=cv.INTER_AREA)

# gets a midpoint and constructs the 4 corners that the advert image should be placed in.
def get_advert_coords_from_midpt(ad_im, midpt):
    axh = ad_im.shape[0] // 2
    ayh = ad_im.shape[1] // 2
    return np.array([
        [midpt[0] - axh, midpt[1] - ayh],
        [midpt[0] + axh, midpt[1] - ayh],
        [midpt[0] + axh, midpt[1] + ayh],
        [midpt[0] - axh, midpt[1] + ayh],
    ])

# Performs perspective transformation of a set of points with a homography matrix.
def perspective_tx_cv(pts, h):
    pts_reshaped = np.float32(pts)[:, np.newaxis, :]
    return cv.perspectiveTransform(pts_reshaped, h).squeeze(1)
```

```
# Fits a source image within a destination image at specified anchor points that may be perspectively
transformed.
def fit_image_in_anchor_pts(source_image, destination_image, anchor_pts):
    src_height, src_width = source_image.shape[:2]
    src_points = np.float32([[0, 0], [src_width, 0], [src_width, src_height], [0, src_height]])
    dst_points = np.float32([anchor_pts[3], anchor_pts[0], anchor_pts[1], anchor_pts[2]])

    homography_matrix = cv.getPerspectiveTransform(src_points, dst_points)
    warped_image = cv.warpPerspective(source_image, homography_matrix, (destination_image.shape[1],
destination_image.shape[0]))

    mask = np.zeros_like(destination_image, dtype=np.uint8)
    cv.fillConvexPoly(mask, dst_points.astype(int), (255, 255, 255))

    mask_inv = cv.bitwise_not(mask)
    destination_bg = cv.bitwise_and(destination_image, mask_inv)
    warped_fg = cv.bitwise_and(warped_image, mask)
    result = cv.add(destination_bg, warped_fg)
    return result



# Computes a homography matrix based on the birds eye view reference of the pitch,
# transforms an advertisement image path passed as an arg to the pitch using homography and anchors it
around a midpoint.
# Rotation, translation and scale of the advert is taken care of by the homography based warping.
def insert_advert(ad_img_path:str, scene_im, point_map:dict, reference_point_map:dict,
anchor_mid_point:tuple, scaling_factor:int = 20):
    ad_im = cv.imread(ad_img_path)
    # reduce the advert image size to fit within image bounds.
    ad_im = rescale_frame(ad_im, scaling_factor)
    # The points in the dictionary gets put into index-ed lists for obect and scene points for homography
    scene_pts = []
    obj_pts = []
    ret = False
    for k, v in reference_point_map.items():
        # reference im (birds eye view of the pitch) contains all points manually marked and labelled
        # in the scene, only a subset of points is visible, this check makes sure invisible pts and
discarded for now.
        if k not in point_map:
            continue
        obj_pts.append(v)
        scene_pts.append(point_map.get(k))
        reference_point_map[k] = point_map.get(k)
    if(len(obj_pts) < 4):
        print("Too low correspondences to use homography")
        return scene_im, ret
    # actual computation of homography
    print(obj_pts)
    print(scene_pts)
    h, status = cv.findHomography(np.array(obj_pts), np.array(scene_pts), cv.RANSAC)
    ad_pts = get_advert_coords_from_midpt(ad_im, anchor_mid_point)
    # perform a perspective transform on the 4 anchor points of the ad to apply homographic warping
    ad_pts_tx = perspective_tx_cv(ad_pts, h)
    # Add the image within the transformed anchor points
    return fit_image_in_anchor_pts(ad_im, scene_im, ad_pts_tx), True

def between(cap, lower: int, upper: int) -> bool:
    return lower <= int(cap.get(cv.CAP_PROP_POS_MSEC)) < upper

cnt = 0
#MAIN
while(videoCapture.isOpened()):
    ret, frame = videoCapture.read()

    if ret == True:
        TestImage = frame.copy()
        TestImageGoal = np.copy(TestImage)
        OrigImage = np.copy(TestImage)

    else:
        break
```

```
    lower_bound = np.array([200, 200, 200], dtype=np.uint8)
    upper_bound = np.array([255, 255, 255], dtype=np.uint8)

    # Create a mask where pixels within the range are white, others are black
    TestImageGoal = cv.inRange(TestImageGoal, lower_bound, upper_bound)
    TestImageGoal = cv.cvtColor(TestImageGoal, cv.COLOR_GRAY2BGR)



    #Step 1: Apply green color segmentation (JANNAT)
    segmented_image = color_segmentation_green(TestImage)

    #TEST masking out the outside area
    filtered_image = delete_outside_area(segmented_image)

    # Step 2: Detect edges using Canny edge detection (JANNAT)
    # goal post/set point stuff
    edges, gray_image = detect_edges(TestImageGoal)
    LineGoal = getMaxLines(edges)

    edges, gray_image = detect_edges(filtered_image)
    LineFiltered = getMaxLines(edges)



    #Get intersection points. Extend lines by 75px and check if they intersect within bounds
    CornerPoints = get_intersection_points(LineFiltered)

    #Compare the points to the previous frame, use only the ones that were there before
    AveragePoints = average_strong_corners(CornerPoints)



    #Print lines
    if LineFiltered:
        #print(LineFiltered)
        for i in range(0, len(LineFiltered)):
            l = LineFiltered[i]
            cv.line(OrigImage, (l[0], l[1]), (l[2], l[3]), color[i].tolist(), 3, cv.LINE_AA)

    # get stable point from pitch, center of crossbar, and length of crossbar
    setPoint, goalPostLength = get_goal_reference(LineGoal)

    # create an empty list to hold the 4D points (original 2D point + distance + octant)
    averagePointsDistance = []
    calc_dist_orient(AveragePoints, averagePointsDistance, setPoint, goalPostLength)

    # sort based on distance from set point
    averagePointsDistance.sort(key=itemgetter(3))
    #print(averagePointsDistance)

    scene_pt_map = {}
    for data in averagePointsDistance:
        scene_pt_map[(int(data[2]), int(data[3]))] = [int(data[0]), int(data[1])]

    print("==================")
    common_pts = 0
    for k, v in scene_pt_map.items():
        if k in REF_PT_MAP:
            common_pts+=1
        print(f"[{k[0]}, {k[1]}] --> {v[0]}, {v[1]}")
    print(f"Common Points : {common_pts}")
    print("==================")

    ad_im, status = insert_advert("./logo.png", OrigImage, scene_pt_map, REF_PT_MAP, scene_pt_map.get((2,
8), [0, 0]))

    if status == True:
        cv.imwrite(f"{str(cnt)}.jpeg", ad_im)
        cnt+=1
```

```
    #print(scene_pt_map)
    mark_pts(OrigImage, averagePointsDistance)

    #Print points
    # if AveragePoints is not None:
    #     #print(AveragePoints)
    #     for i in range(0, len(AveragePoints)):
    #         p = AveragePoints[i]
    #         OrigImage = cv.circle(OrigImage, (p[0], p[1]), 20, (255, 255, 255), -1)

    #cv.imshow("Source1", segmented_image)
    #cv.imshow("Source2", eroded_edges)
    #cv.imshow("Source3", hough_image)
    cv.imshow("Source4", rescale_frame(OrigImage))
    cv.imshow("AD Image", ad_im)
    #cv.imshow("Source5", TestMask)
    #print("Frame")

    #output_frame = OrigImage
    #output.write(output_frame)

    if cv.waitKey(16) & 0xFF == ord('q'):
        break


#Release and destroy
videoCapture.release()
#output.release()
cv.destroyAllWindows()
```

REFERENCES

[1] D. Goldman, "The Evolution of Virtual Advertising in Professional Sports," *The Gettysburgian*, Dec. 15, 2023. [Online]. Available: https://gettysburgian.com/2023/12/the-evolution-of-virtual-advertising-in-professional-sports/.

[2] J. Lee, "Enhancing Viewer Engagement through Virtual Advertising," *Journal of Sports Marketing*, vol. 12, no. 4, pp. 56-70, 2022. [Online]. Available: https://www.examplejournal.com/2022/12/enhancing-viewer-engagement-virtual-advertising/.

[3] M. Smith, "Challenges of Virtual Advertising in Live Sports Broadcasts," *International Journal of Digital Media*, vol. 5, no. 3, pp. 45-62,2023.[Online].Available:https://www.examplejournal.com/2023/08/challenges-virtual-advertising/.